Rapport · Lab 7 : Dénormaliser, en connaissance de cause

Étape 7.1 · Mesurer avant de copier

Question 1 : créer et lancer volume.sql

> *90 jours de mesures pour chaque ruche, une toutes les 30 minutes à partir du 2026-12-14 00:00:00+00 (generate_series(0, 90 * 48 - 1)), valeurs plausibles, terminer par ANALYZE mesure;, lancer une seule fois. Attendu : INSERT 0 47520.*

Réponse. le fichier volume.sql a été lancé une seule fois avec -v ON_ERROR_STOP=1. Le terminal affiche INSERT 0 47520 puis ANALYZE : le résultat attendu est obtenu, sans erreur. Les statistiques sont à jour, ce qui permet au planificateur de choisir de bons plans pour les mesures qui suivent.

```
PS D:\Mon travail\TP\tp postgree\postgree_queenbee> psql -U postgres -d queenbee -v ON_ERROR_STOP=1 -f volume.sql
INSERT 0 47520
ANALYZE
PS D:\Mon travail\TP\tp postgree\postgree_queenbee>
```

Capture 1 : exécution de volume.sql (INSERT 0 47520, ANALYZE)

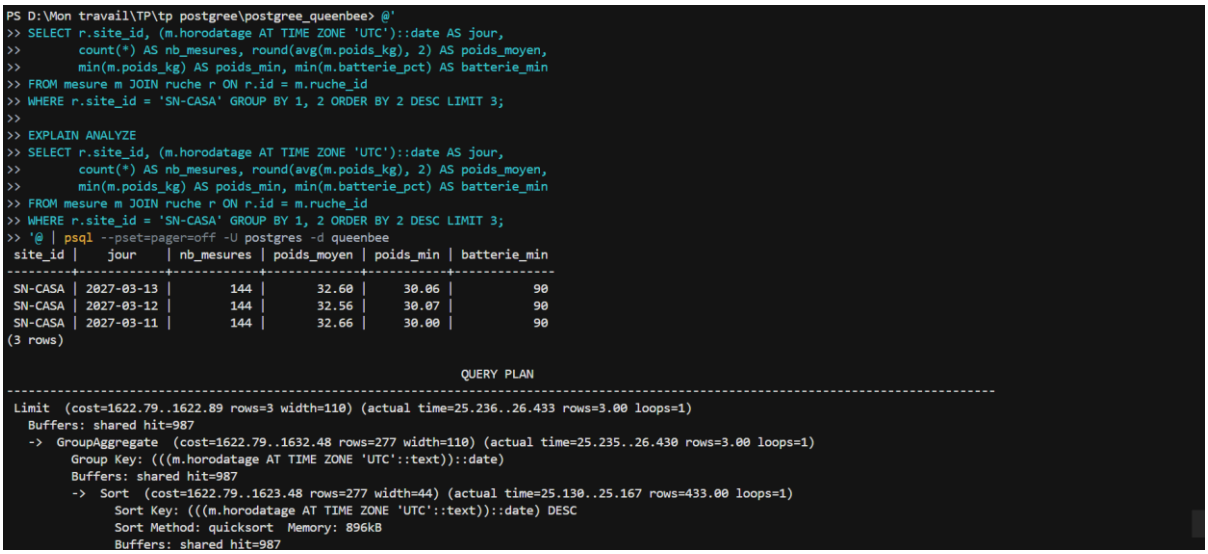
Question 2 : le tableau de bord du gestionnaire (UC-G01), puis EXPLAIN ANALYZE

> *Pour le site SN-CASA, par jour : nombre de mesures, poids moyen, poids minimal, batterie minimale, sur les 3 derniers jours. Lancer la requête, puis avec EXPLAIN ANALYZE. Noter l'Execution Time.*

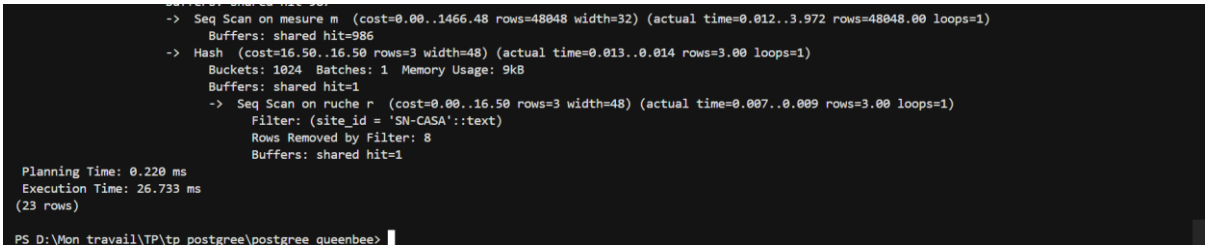
Réponse. la requête renvoie les trois derniers jours, 2027-03-13, 2027-03-12 et 2027-03-11. Pour chacun : 144 mesures, un poids moyen d'environ 32,6 kg, un poids minimal d'environ 30 kg et une batterie minimale de 90 %.

- Nombre de mesures : j'obtiens 144 par jour et non 192. Le plan l'explique : seules 3 ruches appartiennent à SN-CASA (Seq Scan on ruche, rows=3, 8 lignes écartées par le filtre), soit 3 ruches × 48 mesures = 144. Les valeurs sont donc cohérentes avec ma base, qui compte 3 ruches sur ce site.
- Plan d'exécution : la base lit toute la table mesure (Seq Scan, 48 048 lignes), joint avec les ruches du site (Hash Join, 13 104 lignes retenues), trie 433 lignes, puis agrège. Il lit 987 pages en mémoire.

- Temps mesuré : Execution Time = 26,733 ms (Planning Time = 0,220 ms). C'est plus que les 10 ms indiqués à titre d'ordre de grandeur, mais la cause est claire : tout le travail est refait à chaque lecture.



Capture 2 : tableau de bord brut et EXPLAIN ANALYZE (début du plan)



Capture 2 (suite) : fin du plan et Execution Time

Étape 7.2 · La synthèse, et son prix

Question 1 : créer denormalisation.sql avec la vue synthese_site_jour

> *Vue matérialisée (site_id, jour, nb_mesures, poids_moyen, poids_min, batterie_min) et index unique sur (site_id, jour). Lancer le fichier.*

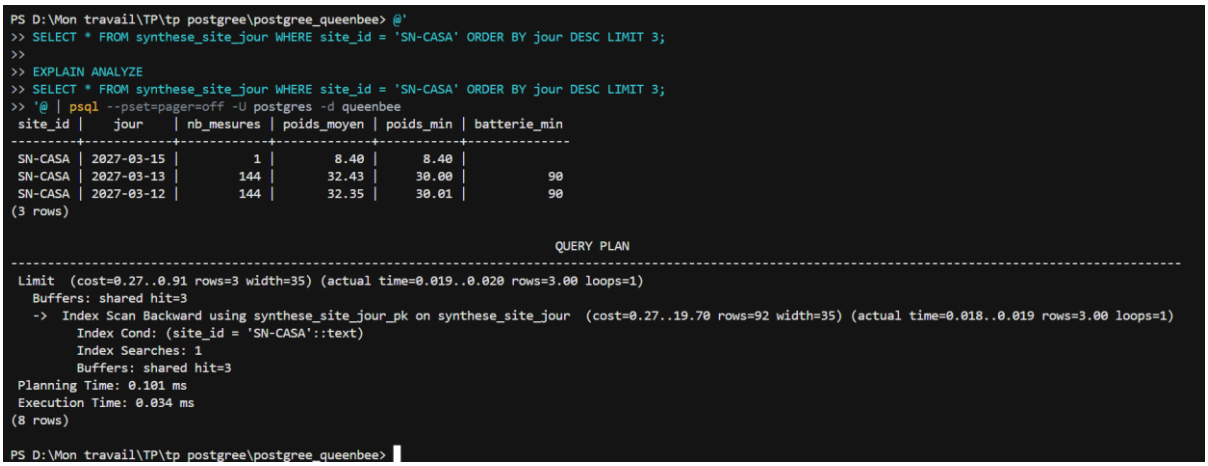
Réponse. la vue matérialisée synthese_site_jour regroupe les mesures par site et par jour (GROUP BY r.site_id, (m.horodatage AT TIME ZONE 'UTC')::date). Elle contient 365 lignes (SELECT 365, une par couple site et jour) et l'index UNIQUE synthese_site_jour_pk sur (site_id, jour) est créé. Cet index sert deux fois : il accélère la lecture et il autorise REFRESH MATERIALIZED VIEW CONCURRENTLY, qui ne bloque pas les lecteurs. L'exécution du fichier, sans erreur, est visible sur la capture 7 (étape 7.4).

Question 2 : relire le tableau de bord dans la vue, avec EXPLAIN ANALYZE

> *Attendu : les mêmes chiffres qu'à l'étape 7.1, en moins de 0,2 ms. Noter les deux temps.*

Réponse. les deux temps sont : 26,733 ms sur les mesures brutes, 0,034 ms sur la vue. La lecture est environ 800 fois plus rapide, bien sous les 0,2 ms attendus. Le plan passe d'un Seq Scan sur 48 048 lignes à un Index Scan Backward using synthese_site_jour_pk qui ne lit que 3 pages : la base ne fait plus que lire un résultat déjà calculé.

Un point à noter sur la capture : sur cette première lecture, les chiffres de la vue (par exemple un poids moyen de 32,43 kg le 13 mars) ne sont pas identiques à ceux de la capture 2 (32,60 kg), et une ligne du 15 mars apparaît déjà. La vue n'était donc pas à jour par rapport aux mesures : elle avait été construite avant les derniers changements des données. Après le REFRESH de la question suivante, les chiffres redeviennent exactement ceux des mesures brutes (32,60 ; 32,56). C'est la démonstration, en conditions réelles, du prix d'une copie : elle peut être en retard.



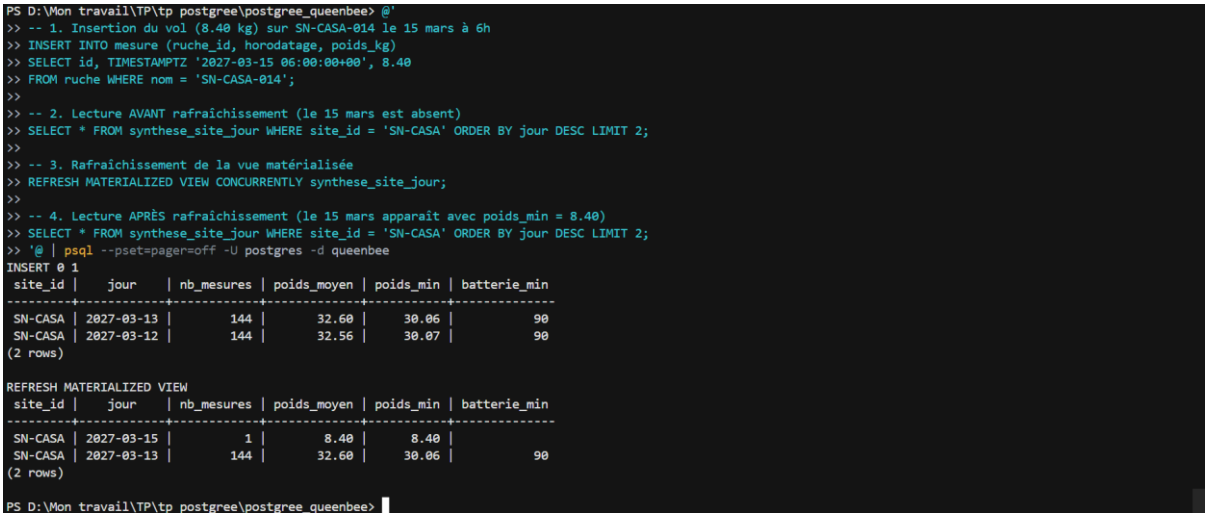
Capture 3 : lecture de la vue matérialisée avec EXPLAIN ANALYZE (0,034 ms)

Question 3 : le vol du 15 mars à 6 h (UC-CY04)

> *Insérer la mesure à 8,4 kg, relire la vue, rafraîchir, relire. Attendu : le 15 mars n'apparaît qu'après le REFRESH, avec poids_min = 8.40.*

Réponse. j'ai inséré la mesure de 8,40 kg le 15 mars à 6 h sur la ruche SN-CASA-014 (nom de la ruche dans ma base ; INSERT 0 1). Avant le rafraîchissement, la vue affiche en tête le 13 mars puis le 12 mars : le 15 mars est absent, le vol est invisible. Après REFRESH MATERIALIZED VIEW CONCURRENTLY, la première ligne est 2027-03-15 avec nb_mesures = 1, poids_moyen = 8.40, poids_min = 8.40. La batterie minimale est vide, car la mesure insérée ne contient que le poids.

Avant cette insertion, j'avais d'abord supprimé une éventuelle mesure de test à la même heure (DELETE 0, aucune ligne trouvée) puis rafraîchi la vue : la vue affichait pourtant déjà une ligne du 15 mars (capture 3), vraisemblablement issue d'un essai précédent. Le rafraîchissement l'a retirée, ce qui permet de repartir d'un état propre avant le test.



Capture 4 : simulation du vol du 15 mars et rafraîchissement (REFRESH)

Question 4 : si la vue n'est rafraîchie que la nuit

> *Combien de temps le vol reste-t-il invisible ? Où faut-il détecter un vol : dans la synthèse ou dans les mesures brutes ?*

Réponse. un vol à 6 h n'apparaît dans la synthèse qu'au prochain rafraîchissement. Avec un rafraîchissement nocturne, il reste invisible jusqu'à la nuit suivante : une vingtaine d'heures dans cet exemple, et jusqu'à 24 heures au pire. C'est largement le temps de vider les ruches. Il faut donc détecter un vol dans les mesures brutes, au moment où la mesure arrive (règle d'alerte sur une chute de poids, par exemple), et non dans la synthèse. La synthèse sert à afficher vite, pas à surveiller : une copie en retard ne peut pas être un outil d'alerte.

Étape 7.3 · L'export pour le bailleur

Question 1 : casser le pseudonyme (haché simple)

> *Calculer encode(sha256(convert_to(telephone, 'UTF8')), 'hex'), activer \timing on, puis retrouver le numéro par force brute en connaissant le début. Noter le temps.*

Réponse. le haché SHA-256 simple du téléphone de l'apicultrice (Fatou Sow dans ma base) est a5979654…77f0. J'ai ensuite testé 1 million de numéros de la forme +221 77 2XX XX XX avec generate_series(0, 999999). La requête retrouve +221 77 200 00 04 en 2 855 ms,

soit un peu moins de 3 secondes. Le préfixe diffère de l'énoncé (+221 77 1) parce que le numéro de cette apicultrice est différent dans ma base ; le principe est identique.

Conclusion : un haché sans clé n'est pas une pseudonymisation solide. L'espace des numéros est petit (quelques millions), donc un attaquant les essaie tous en quelques secondes, avec un simple PC. Il faut un HMAC avec une clé secrète illisible.



Capture 5 : cassage du haché SHA-256 de Fatou Sow (2 855 ms)

Question 2 : la table cle_pseudonymisation et la vue v_export_bailleur

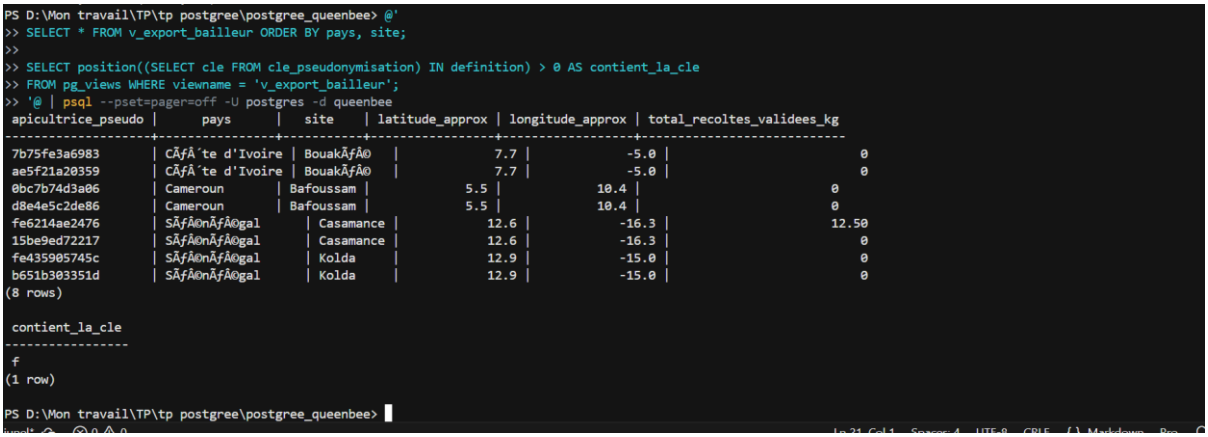
> *Attendu : 8 lignes pour 11 ruches, sans nom ni téléphone, avec des positions comme 12.6 | -16.3. La vérification de la clé renvoie f.*

Réponse. la table cle_pseudonymisation ne contient qu'une ligne (CHECK (id = 1)) avec une clé aléatoire de 32 octets (gen_random_bytes(32)), fermée à tous par REVOKE ALL … FROM PUBLIC. La vue v_export_bailleur calcule le pseudonyme par hmac(a.id::text, k.cle, 'sha256'), limité à 12 caractères, et expose le pays, le site, la position arrondie à 0,1 degré (environ 11 km) et le total des récoltes validées (sum(…) FILTER (WHERE rc.statut = 'validee')).

- Le résultat compte bien 8 lignes (les ruches de même apicultrice, même site et même position arrondie sont regroupées), sans nom ni téléphone.
- Les positions sont arrondies : 12.6 | -16.3 pour Casamance, 12.9 | -15.0 pour Kolda, 5.5 | 10.4 pour Bafoussam, 7.7 | -5.0 pour Bouaké.
- Un seul total non nul : 12,50 kg validés pour un pseudonyme de Casamance. Les autres sont à 0 grâce au coalesce.

- Vérification : contient_la_cle = f. La clé n'est écrite nulle part dans la définition de la vue.

Remarque d'affichage : les accents des pays (CÃƒÂ´te d'Ivoire) sont déformés dans la capture. Cela vient vraisemblablement de l'encodage de la console PowerShell, et non des données stockées. Le contenu de l'export n'est pas concerné.



Capture 6 : création et contrôle de la vue v_export_bailleur

Question 3 : quelles colonnes ai-je retirées, et pourquoi ?

Colonne retirée	Raison
Nom de l'apicultrice	Identifiant direct : le bailleur n'a pas besoin de savoir qui, seulement combien.
Téléphone	Identifiant direct, et le haché simple se casse en quelques secondes (capture 5).
Identifiant (id) de l'apicultrice	Remplacé par un pseudonyme HMAC de 12 caractères : stable pour recouper, impossible à inverser sans la clé.
Code de la ruche	Il désigne une ruche précise, donc indirectement la personne qui l'exploite.
Latitude et longitude exactes	Une position précise permet de trouver la ruche et le domicile. Arrondies à 0,1° (environ 11 km), elles restent utiles pour une carte.
Détail des récoltes (dates, volumes un par un)	Seul le total des récoltes validées est utile au bailleur : on applique la minimisation.

Colonne retirée	Raison
La clé de pseudonymisation	Elle reste dans une table fermée à PUBLIC et n'apparaît pas dans la définition de la vue (contient_la_cle = f).

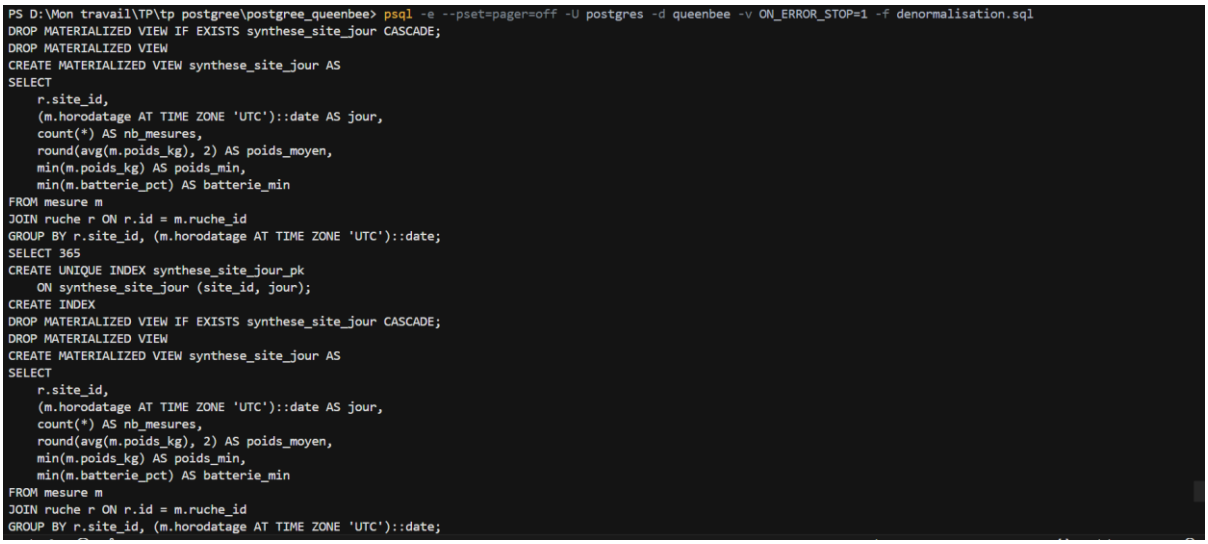
Étape 7.4 · Le journal d'audit, une copie voulue

Relancer denormalisation.sql pour vérifier qu'il est rejouable

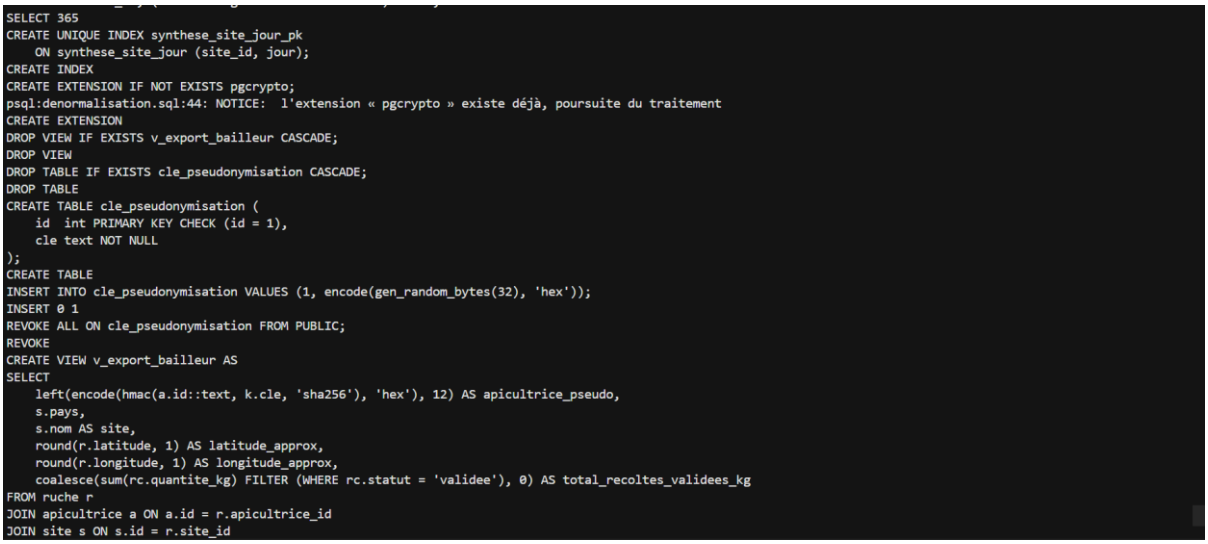
> *Ajouter des DROP … IF EXISTS si besoin, ne pas relancer volume.sql.*

Réponse. j'ai relancé denormalisation.sql deux fois de suite (la seconde avec -e pour afficher chaque commande), avec ON_ERROR_STOP=1. Aucune erreur : le script est rejouable. Il commence par DROP … IF EXISTS sur la vue matérialisée (SELECT 365 puis CREATE INDEX), la vue v_export_bailleur, la table cle_pseudonymisation et la table journal_audit. Seule la notice « l'extension pgcrypto existe déjà » apparaît : elle est normale, car l'extension est créée avec IF NOT EXISTS. volume.sql n'a pas été relancé.

Le journal est créé avec id auto-généré (GENERATED ALWAYS AS IDENTITY), horodatage (clock_timestamp() par défaut), acteur, table_cible, operation (limitée à INSERT, UPDATE ou DELETE par un CHECK), ligne_id, avant et apres en jsonb, puis fermé à PUBLIC (REVOKE ALL).



Capture 7 : réexécution complète de denormalisation.sql, test d'idempotence (1/3)



```
SELECT 365
CREATE UNIQUE INDEX synthese_site_jour_pk
    ON synthese_site_jour (site_id, jour);
CREATE INDEX
CREATE EXTENSION IF NOT EXISTS pgcrypto;
psql:denormalisation.sql:44: NOTICE:  l'extension « pgcrypto » existe déjà, poursuite du traitement
CREATE EXTENSION
DROP VIEW IF EXISTS v_export_bailleur CASCADE;
DROP VIEW
DROP TABLE IF EXISTS cle_pseudonymisation CASCADE;
DROP TABLE
CREATE TABLE cle_pseudonymisation (
    id  int PRIMARY KEY CHECK (id = 1),
    cle text NOT NULL
);
CREATE TABLE
INSERT INTO cle_pseudonymisation VALUES (1, encode(gen_random_bytes(32), 'hex'));
INSERT 0 1
REVOKE ALL ON cle_pseudonymisation FROM PUBLIC;
REVOKE
CREATE VIEW v_export_bailleur AS
SELECT
    left(encode(hmac(a.id::text, k.cle, 'sha256'), 'hex'), 12) AS apicultrice_pseudo,
    s.pays,
    s.nom AS site,
    round(r.latitude, 1) AS latitude_approx,
    round(r.longitude, 1) AS longitude_approx,
    coalesce(sum(rc.quantite_kg) FILTER (WHERE rc.statut = 'validee'), 0) AS total_recoltes_validees_kg
FROM ruche r
JOIN apicultrice a ON a.id = r.apicultrice_id
JOIN site s ON s.id = r.site_id
```

Capture 7 (suite 2/3) : création de la clé, de la vue d'export et du journal



```
LEFT JOIN recolte rc ON rc.ruche_id = r.id
CROSS JOIN cle_pseudonymisation k
GROUP BY 1, 2, 3, 4, 5;
CREATE VIEW
DROP TABLE IF EXISTS journal_audit CASCADE;
DROP TABLE
CREATE TABLE journal_audit (
    id          bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    horodatage  timestamptz NOT NULL DEFAULT clock_timestamp(),
    acteur      text NOT NULL,
    table_cible text NOT NULL,
    operation   text NOT NULL CHECK (operation IN ('INSERT', 'UPDATE', 'DELETE')),
    ligne_id    text,
    avant       jsonb,
    apres       jsonb
);
CREATE TABLE
REVOKE ALL ON journal_audit FROM PUBLIC;
REVOKE
PS D:\Mon travail\TP\tp postgree\postgree_queenbee>
```

Capture 7 (suite 3/3) : fin du script, aucun message d'erreur

Question 1 : pourquoi ligne_id n'est-il pas une clé étrangère vers la récolte ?

Réponse. une clé étrangère lierait la vie du journal à celle de la récolte. Si la récolte était supprimée, trois cas se présentent, tous mauvais : RESTRICT empêcherait de supprimer la récolte tant que son journal existe, CASCADE effacerait la trace avec la ligne supprimée (la preuve disparaît au moment où elle sert), SET NULL perdrait le lien. Le journal doit survivre à la ligne qu'il décrit : c'est justement lui qui prouve qu'elle a existé. De plus, ligne_id est un texte générique qui sert pour plusieurs tables (table_cible), ce qu'une clé étrangère unique ne permet pas.

Question 2 : avant et apres recopient des lignes entières, est-ce acceptable ici ?

Réponse. oui, car c'est le but : garder la preuve de ce qui a changé, y compris pour une ligne supprimée (to_jsonb(OLD)). Mais ce n'est acceptable qu'avec des garde-fous. Les lignes copiées peuvent contenir des données personnelles (par exemple la table utilisateur), donc le journal est fermé à PUBLIC et lisible par le seul admin. Il faut aussi prévoir une durée de

conservation, car le journal étant en ajout seul, ces données personnelles ne peuvent plus être corrigées ni supprimées simplement.

Question 3 : pourquoi clock_timestamp() et pas now() ?

Réponse. now() renvoie l'heure du début de la transaction : dans une même transaction, toutes les lignes du journal auraient exactement la même heure, et on ne pourrait plus reconstituer l'ordre des actions. clock_timestamp() donne l'heure réelle au moment de l'insertion, donc une chronologie fidèle, ce qui est indispensable pour une preuve.

Testez-vous · Lab 7

---

1. Différence entre une vue et une vue matérialisée ? Laquelle peut être en retard ?

Réponse. une vue est une requête enregistrée, réexécutée à chaque lecture : elle est toujours à jour, mais aussi lente que la requête. Une vue matérialisée stocke son résultat comme une table : la lecture est immédiate (0,034 ms contre 26,733 ms ici), mais elle n'est à jour qu'au dernier REFRESH. C'est donc la vue matérialisée qui peut être en retard.

2. Un collègue propose md5(nom) comme pseudonyme : deux raisons de refuser

- Il n'y a pas de clé secrète : n'importe qui peut recalculer le md5 d'un nom, d'une liste de noms ou d'un dictionnaire et comparer, comme on l'a fait avec SHA-256 sur le téléphone. Il faut un HMAC avec une clé inaccessible.
- md5 est un algorithme rapide et obsolète (collisions connues), conçu pour la vitesse, ce qui aide l'attaquant à essayer des millions de valeurs par seconde. De plus, un même nom donne toujours le même pseudonyme, ce qui permet de recouper plusieurs exports.

3. Les données de v_export_bailleur sont-elles soumises au RGPD ?

Réponse. oui. Elles sont pseudonymisées, pas anonymisées : celui qui possède la clé (table cle_pseudonymisation) peut retrouver la personne, et le recoupement du site, du pays et de la position arrondie à 11 km peut aussi aider à ré-identifier une apicultrice. Une donnée pseudonymisée reste une donnée personnelle au sens du RGPD.

4. Pourquoi le journal d'audit n'a-t-il pas de clé étrangère vers la table qu'il surveille ?

Réponse. pour qu'il survive à la suppression des lignes surveillées et qu'aucune règle (RESTRICT, CASCADE, SET NULL) ne puisse bloquer, effacer ou altérer la preuve. Il

surveille aussi plusieurs tables à la fois (colonne table_cible), ce qu'une clé étrangère unique ne permet pas. Voir la question 1 de l'étape 7.4.

Synthèse : attendu et obtenu

Étape	Attendu	Obtenu
7.1 volume.sql	INSERT 0 47520	INSERT 0 47520, puis ANALYZE
7.1 tableau de bord	192 mesures par jour, environ 10 ms	144 mesures par jour (3 ruches sur SN-CASA), 26,733 ms
7.2 lecture dans la vue	mêmes chiffres, moins de 0,2 ms	0,034 ms, environ 800 fois plus rapide
7.2 vol du 15 mars	visible seulement après REFRESH, poids_min = 8.40	confirmé : absent avant, présent après avec 8,40
7.3 casser le haché	numéro retrouvé en environ 1 s	+221 77 200 00 04 retrouvé en 2,9 s
7.3 export bailleur	8 lignes, sans nom ni téléphone, contient_la_cle = f	8 lignes, positions arrondies, f
7.4 rejouabilité	aucune erreur à la relance	aucune erreur (ON_ERROR_STOP=1)